

Al Mahmud Siam

Lecturer, Department of CSE

Z.H.Sikder University of Science and Technology

CSE 4454: Digital Image Processing Lab

Laboratory Experiment 01

Title : Introduction to Digital Image Processing Using Python and OpenCV: Image Reading, Displaying, and Basic Pixel Analysis

1. Objectives

After completing this experiment, students will be able to:

1. Understand the basic concept of a digital image.
2. Understand pixels and pixel intensity values.
3. Set up and use Python with OpenCV and Matplotlib.
4. Read an image using OpenCV.
5. Display an image using Matplotlib.
6. Understand the difference between BGR and RGB color formats.
7. Determine the dimensions of an image.
8. Access individual pixel values.
9. Convert a color image into a grayscale image.
10. Understand that a digital image is represented as numerical data.

2. Introduction to Digital Image Processing

Digital Image Processing (DIP) is the process of using computers to manipulate and analyze digital images.

Examples of image-processing applications include:

- Improving image quality
- Removing noise
- Detecting edges
- Segmenting objects
- Object recognition
- Medical image analysis
- Face detection

- Satellite image analysis
- Image classification

A very important concept in digital image processing is:

A digital image is represented by numerical values.

A computer does not see an image in the same way as a human. Humans see objects, colors, shapes, and patterns. A computer processes the image as a collection of **pixels and numerical values**.

3. What is a Pixel?

A **pixel** (picture element) is the smallest individual element of a digital image.

For example, a grayscale image may be represented as:

```
0  50  100  150
20  80  130  200
40  90  180  255
```

Each number represents the intensity of a pixel.

For an 8-bit grayscale image:

0 → Black
255 → White

Values between 0 and 255 represent different shades of gray.

Therefore:

A grayscale image can be represented as a matrix of intensity values.

4. Color Images

A color image usually contains three color channels:

R → Red
G → Green
B → Blue

Each pixel therefore contains three numerical values.

For example:

(R, G, B)

A pixel might contain:

(128, 200, 50)

which represents a particular color.

5. Software and Libraries Required

In this laboratory, we will use:

- **Python**
- **OpenCV**
- **NumPy**
- **Matplotlib**

OpenCV

OpenCV is an open-source computer vision and image-processing library.

We will use OpenCV for tasks such as:

- Reading images
- Writing images
- Image conversion
- Resizing
- Filtering
- Edge detection
- Thresholding
- Morphological operations

Matplotlib

Matplotlib is a Python visualization library. In this experiment, it will mainly be used to display images.

6. Installing Required Libraries

If OpenCV and Matplotlib are not installed, execute:

pip install opencv-python matplotlib numpy

For Ubuntu, if necessary, Python Tkinter can be installed using:

sudo apt install python3-tk

Code:

```
import cv2
```

```
import matplotlib
```

```
matplotlib.use("TkAgg")
```

```
import matplotlib.pyplot as plt
```

```
image = cv2.imread("image.jpeg")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

plt.imshow(image_rgb)
# plt.axis("off")
plt.show()
print(type(image))
print(image.shape)
print(image[100, 100])
```

7. Checking Image Dimensions

We can determine the dimensions of the image using:

```
print(image.shape)
```

For example, the output might be:

(720, 1280, 3)

720 → Height

1280 → Width

3 → Color channels

Image size = 720×1280 pixels

Number of channels = 3

The three channels are:

Blue

Green

Red

because **OpenCV** uses **BGR** ordering when reading color images.

10. Understanding BGR

One important feature of OpenCV is that it normally reads color images in:

BGR

rather than:

RGB

Therefore:

B → Blue
G → Green
R → Red

```
print(image[100, 100])
```

we may get:

```
[148 131 128]
```

This represents the pixel located at:

Row = 100
Column = 100

The values are:

Blue = 148
Green = 131
Red = 128

```
image[100,100] = [148, 131, 128]
```

11. Understanding Pixel Coordinates

The following command:

```
image[100, 100]
```

means:

```
image[row, column]
```

Therefore:

```
image[100, 100]
```

means:

Access the pixel at row 100 and column 100.

Python uses **zero-based indexing**, so the first pixel is:

```
image[0, 0]
```

The next pixel in the same row is:

```
image[0, 1]
```

and so on.

12. Displaying an Image

We can display the image using Matplotlib.

However, because OpenCV uses BGR and Matplotlib expects RGB, we first need to convert the image.

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

Explanation

`cv2.cvtColor()` is used to convert an image from one color space/format to another.

Here:

```
cv2.COLOR_BGR2RGB
```

means:

BGR → RGB

The converted image is stored in:

```
image_rgb
```

13. Displaying the Converted Image

Now we can display it:

```
plt.imshow(image_rgb)
plt.axis("off")
plt.show()
```

Explanation

```
plt.imshow(image_rgb)
```

displays the image.

```
plt.axis("off")
```

removes the coordinate axes from the displayed image.

```
plt.show()
```

opens/displays the image.

14. Converting Color Image to Grayscale

A grayscale image contains only one intensity value for each pixel.

We can convert the color image to grayscale using:

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Now check the dimensions:

```
print(gray.shape)
```

For example:

```
(720, 1280)
```

Notice that the 3 is gone. Because a grayscale image has only **one channel**.

Color

Pixel = [B, G, R]

Example:

```
[148, 131, 128]
```

3 values per pixel

Grayscale

Pixel = Intensity

Example:

```
133
```

1 value per pixel

17. Displaying the Grayscale Image

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.imshow(gray, cmap="gray")
plt.axis("off")
plt.show()

cmap="gray"
```

18. Accessing a Grayscale Pixel

```
print(gray[100, 100])
```

Instead of three values such as:

```
[148 131 128]
```

we will get a single value, for example:

```
133
```

This represents the intensity of that pixel.

For an 8-bit grayscale image:

0 → Black

255 → White

For example:

0 → Black

50 → Dark gray

128 → Medium gray
200 → Light gray
255 → White

19. Modifying Pixel Values

One of the most important ideas in Digital Image Processing is that we can manipulate the pixel values.

For example:

```
gray[100, 100] = 255
```

This changes the pixel at row 100, column 100 to white.

We can also modify a rectangular region:

```
gray[100:200, 100:200] = 0
```

This sets a region of pixels to black.

Display the result:

```
plt.imshow(gray, cmap="gray")  
plt.axis("off")  
plt.show()
```

20. Basic Image Operations

Some basic operations that will be useful throughout this course are:

Reading

```
image = cv2.imread("image.jpg")
```

Displaying

```
plt.imshow(image_rgb)  
plt.show()
```

Checking dimensions

```
print(image.shape)
```

Accessing a pixel

```
print(image[100, 100])
```

Converting to grayscale

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Resizing

```
resized = cv2.resize(image, (400, 300))
```

Cropping

```
crop = image[100:400, 100:500]
```

Saving

```
cv2.imwrite("output.jpg", gray)
```

21. Complete Demonstration Program

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
# -----
```

```
# 1. Read the image
```

```
# -----
```

```
image = cv2.imread("image.jpg")
```

```
# -----
```

```
# 2. Display image information
```

```
# -----
```

```
print("Image type:", type(image))
```

```
print("Image shape:", image.shape)
```

```
# -----
```

```
# 3. Display a pixel value
```

```
# -----
```

```
print("Pixel at (100,100):", image[100, 100])
```

```
# -----
```

```
# 4. Convert BGR to RGB
```

```
# -----
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# -----
```

```
# 5. Display the original image
```

```
# -----
```

```
plt.imshow(image_rgb)
```

```
plt.title("Original Image")
```

```
plt.axis("off")
```

```
plt.show()
```

```
# -----
```

```
# 6. Convert to grayscale
```

```
# -----
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
print("Grayscale shape:", gray.shape)
```

```
print("Grayscale pixel at (100,100):", gray[100, 100])
```

```
# -----  
# 7. Display grayscale image  
# -----  
plt.imshow(gray, cmap="gray")  
plt.title("Grayscale Image")  
plt.axis("off")  
plt.show()
```

23. Laboratory Exercises

Exercise 1

Read an image and print:

1. Image type
2. Image height
3. Image width
4. Number of channels

Exercise 2

Print the pixel values at:

```
image[0, 0]  
image[100, 100]  
image[200, 200]
```

Observe how the values change.

Exercise 3

Convert the image into grayscale and print:

```
gray.shape
```

Compare it with:

```
image.shape
```

Explain why the dimensions are different.

Exercise 4

Change a rectangular region of the grayscale image to black:

```
gray[100:200, 100:200] = 0
```

Display the result.

Exercise 5

Change another region to white:

```
gray[200:300, 200:300] = 255
```

Display the result.

Exercise 6

Resize the image:

```
resized = cv2.resize(image, (300, 300))
```

Display the resized image.

Exercise 7

Crop a portion of the original image:

```
crop = image[100:400, 100:500]
```

Display the cropped image.

24. Viva Questions

Basic Questions

1. What is Digital Image Processing?
2. What is a digital image?
3. What is a pixel?
4. What is the difference between a pixel and an image?
5. How does a computer represent an image?
6. What is image resolution?
7. What is grayscale?
8. What is RGB?
9. What are the three channels of an RGB image?

OpenCV Questions

10. What is OpenCV?
11. What is the purpose of cv2.imread()?
12. What does cv2.cvtColor() do?
13. Why do we use cv2.COLOR_BGR2RGB?
14. What does image.shape return?
15. How can you access a particular pixel?
16. What does image[100,100] mean?
17. What is the purpose of plt.imshow()?
18. What does plt.show() do?

Conceptual Questions

19. Why does a color image have three values per pixel?
20. Why does a grayscale image have only one value per pixel?
21. What does [148, 131, 128] represent?
22. What do 0 and 255 represent in an 8-bit grayscale image?
23. Why is an image stored as a NumPy array?
24. What happens if you change the value of a pixel?
25. Why does OpenCV use BGR instead of RGB?

25. Short Lab Assignment

Choose any image and perform the following operations:

1. Read the image.
2. Display the image.
3. Print its dimensions.
4. Print the pixel value at (100,100).
5. Convert the image to grayscale.
6. Print the grayscale image dimensions.
7. Display the grayscale image.
8. Resize the image to 300×300 .
9. Crop a portion of the image.
10. Save the grayscale image as gray_output.jpg.

Submission

Students should submit:

- Python source code
- Original image
- Output images
- Short observation/result